Algoritmi paraleli si distribuiți

8. Algoritmi de înmulțire a matricilor

+ openACC

Tipuri de paralelism (review)

- Paralelismul datelor
  - Toate procesoarele execută același cod pe date diferite
- Paralelismul task-urilor
  - Procesoarelor le sunt alocate task-uri care execută cod diferit
- Modele de execuție în paralel
  - Paralelismul datelor
  - Pipeline (i.e., producător-consumator)
  - Graf de task-uri
  - Pool de task-uri

Paralelismul datelor

- Datele/domeniul este descompus și alocat procesoarelor
- Procesoarele execută task-uri identice (sau foarte similare) pe datele primite (care sunt diferite de la un procesor la altul)
- Task-urile se execută în paralel
- Echilibrarea încărcării este obținută prin partiționarea datelor
  - Cantități egale de date sunt alocate tuturor procesoarelor
- Scalabilitatea paralelismului datelor
  - Gradul de paralelizare tinde să crească odată cu creșterea dimensiunii problemei
  - Face ca algoritmii sa fie eficienți/performanți
- SPMD (en., Single Program Multiple Data)
  - Mod oportun de implementare a paralelismului datelor
  - Este asociat cu execuția în paralel cu memorie distribuită

Înmulțirea unei matrici cu un vector

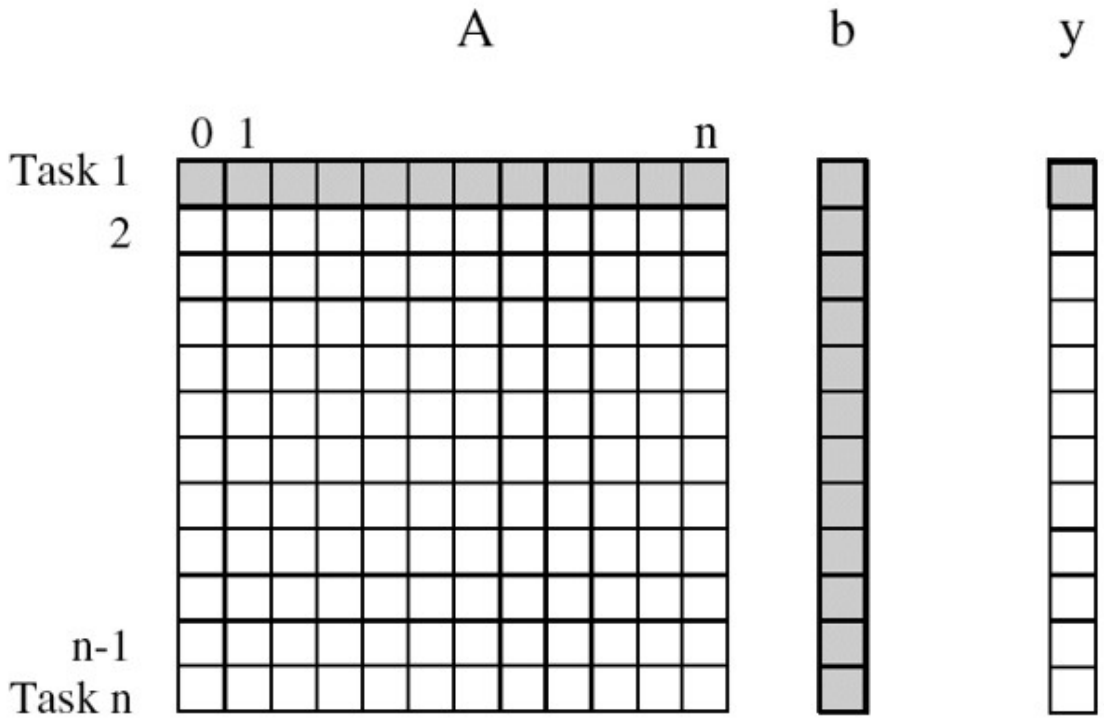
A[nxn], **b**[nx1], **y**[nx1]

A x b = y

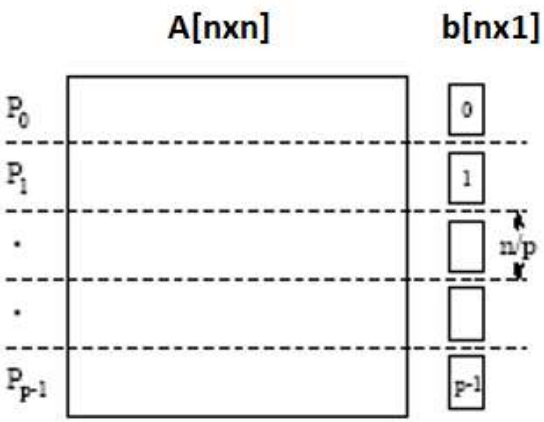
- Creăm ***n*** task-uri, cate unul pentru fiecare rând din **A**.
- Fiecărui task **i** se tranmite (en., *broadcast*) vectorul ***b***.

$$\mathrm{y[i]} = \sum_{i} A[i,j] * b[j]$$

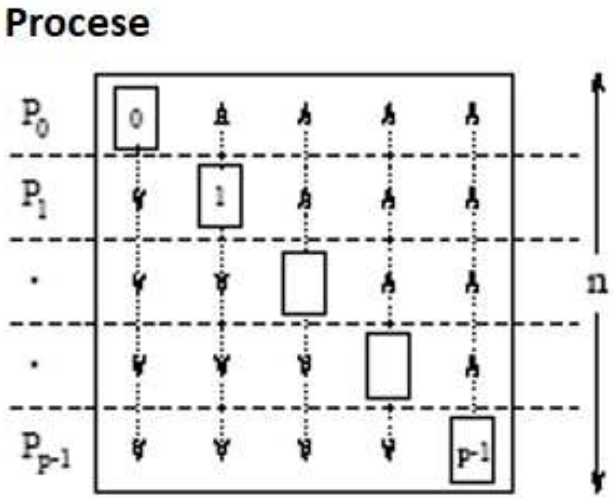
- Fiecare task, determină un element din ***y*** în mod independent.



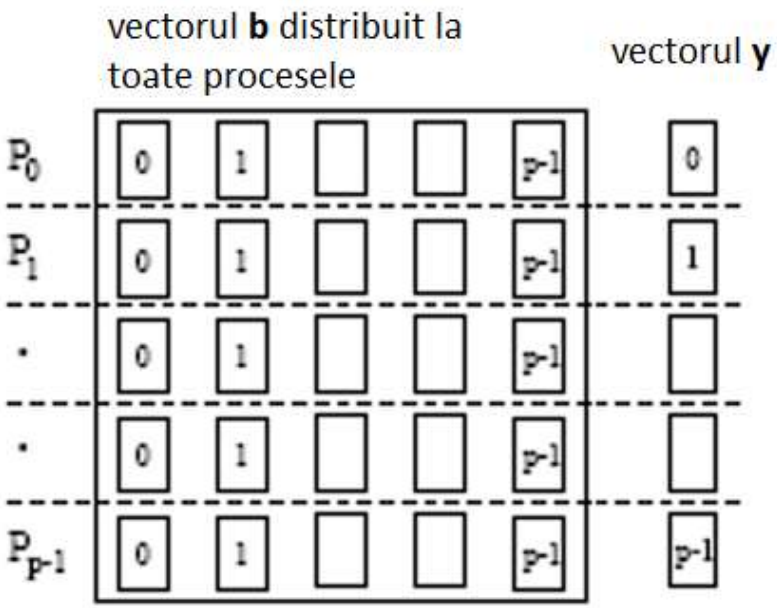
Înmulțirea unei matrici cu un vector (cont.)



Partiționarea inițială a matricii A și vectorul inițial b.
p=n (numărul de procesoare este același cu dimensiunea matricii a)



Distribuirea vectorului b (en., broadcast) către toate procesele.



după transmitere și calcul obținem rezultatul **y**

Înmulțirea unei matrici cu un vector (cont.)

- Vectorul **b** trebuie distribuit către toate matricile.
- Fiecare proces $\mathbf{P_i}$ efectuează următorul calcul

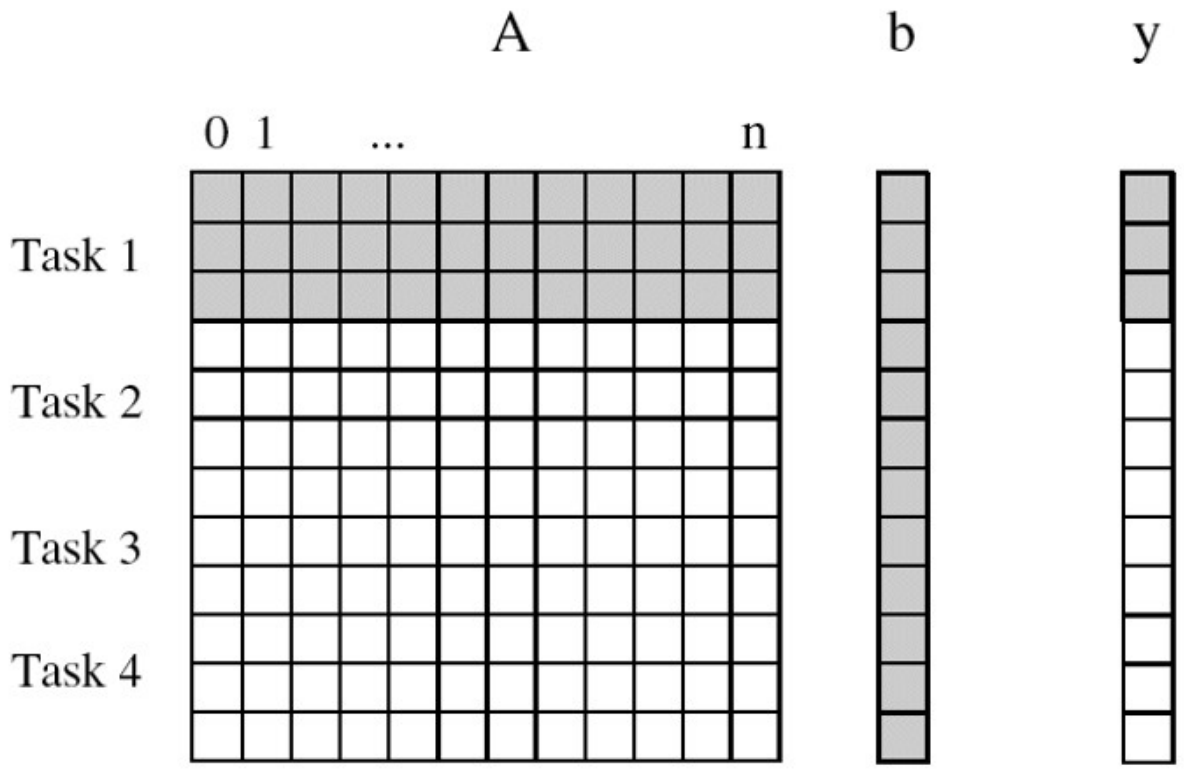
$$y[i] = \sum_{j=0}^{n-1} (A[i,j] * b[j])$$

- Transmiterea vectorului și determinarea valorii lui *y[i]* durează fiecare **O(n)**. Având în vedere că avem la dispoziție **p (=n)** procesoare, înseamă că timpul de execuție paralel este **O(n)**.

Înmulțirea unei matrici cu un vector (cont.)

Dacă avem un număr limitat de task-uri (ex., 4)?

- Care sunt dependințele?
- Care este accelerarea?



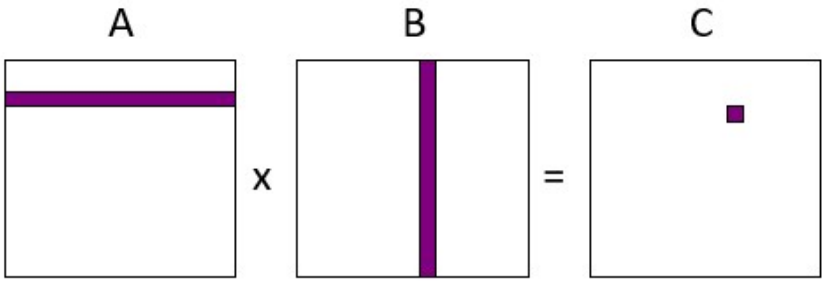
Înmulțirea matricilor

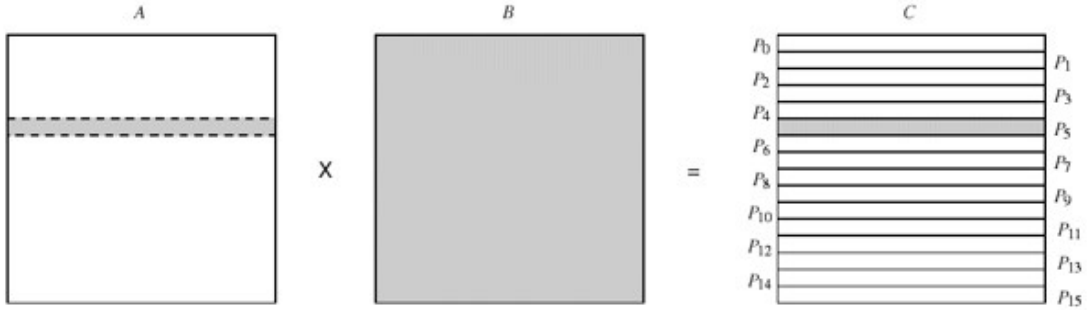
C [nxn] = **A**[nxn] x **B**[nxn]

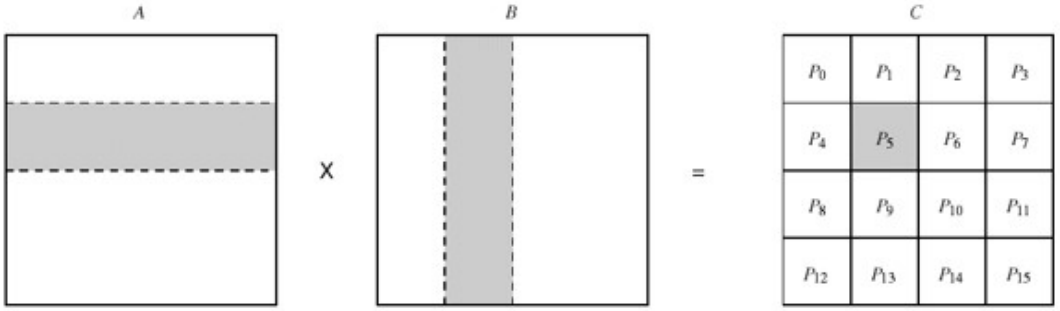
- Complexitatea secvențială este **O(n³)**

Obs: Algoritmul lui Strassen este mai bun decât O(n^3). Se poate folosi ca și kernel serial în cadrul algoritmilor paraleli complecși.

- Partiționare la nivel de rând
  - N task-uri
- Partiționare la nivel de bloc
  - N * N/B task-uri
- Partea gri arată partajarea datelor din matricea B







Înmulțirea matricilor

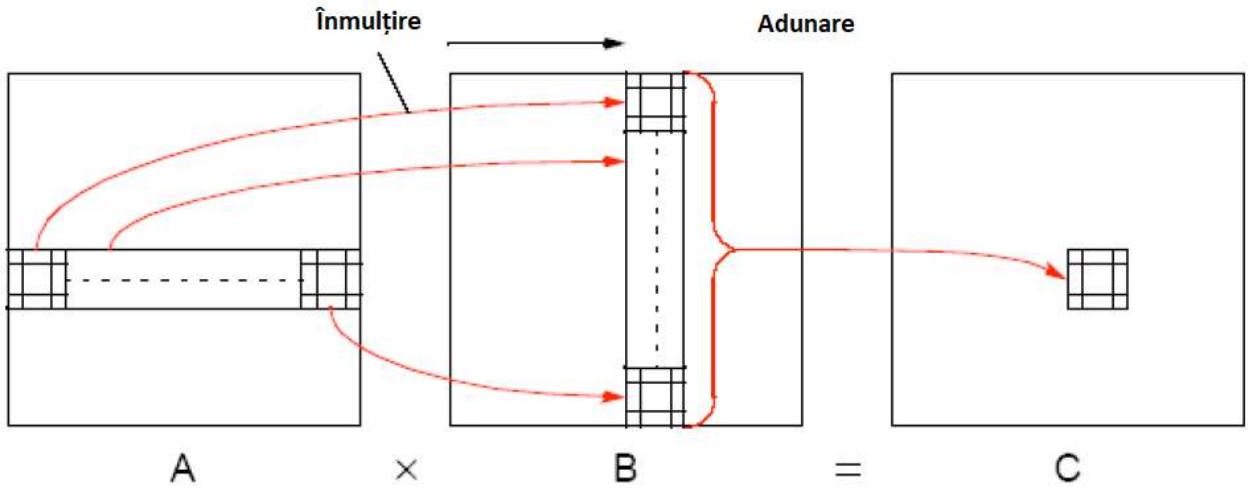
- **Concept util**: efectuarea operațiilor la nivel de *bloc*.
- Împarțim pe A si B în **p** blocuri pătrate $\mathbf{A_{i,j}}$ și $\mathbf{B_{i,j}}$ unde $0 \leq i, j < \sqrt{p}$ de dimensiune [$(n/\sqrt{p}) \times (n/\sqrt{p})$] fiecare.
  - Dacă avem p = 16 procesoare => blocurile au dimensiune (n/4 x n/4) fiecare.
- Procesorul $\mathbf{p_{i,j}}$ ***inițial*** primește/deține $\mathbf{A_{i,j}}$ și $\mathbf{B_{i,j}}$ și determină blocul $\mathbf{C_{i,j}}$ din matricea rezultat.
- ***Observații***:
  - Determinarea submatricii $\mathbf{C_{i,j}}$ necesită **TOATE** submatricile $\mathbf{A_{i,k}}$ și $\mathbf{B_{k,j}}$ pentru pentru $0 \leq k < \sqrt{p}$
  - Este nevoie de tansmiterea catre procesoare (i.e., *broadcast*) a tuturor blocurilor de pe rândurile din A și de pe coloanele din B pentru detrminarea locală a blocului C corespunzător.
  - Trebuie să efectuăm q^3 ($q=n/\sqrt{p}$) înmulțiri de matrici de dimensiune [$(n/q) \times (n/q)$]

Înmulțirea matricilor

Algoritm:

- Se transmit (i.e., broadcast) blocurile corespunzatoare din A (pe rânduri) către procese
- Se transmit (i.e., broadcast) blocurile corespunzatoare din B (pe coloane) către procese
- Fiecare proces $P_{i,j}$ efectueaza calculul

$$C_{i,j} = \sum_{k=0}^{\sqrt{p}-1} A_{i,k} * B_{k,j}$$



Înmulțirea matricilor

- Accelerarea și eficiența:

$$S_p = N^3 / (N^3/p) = p$$

$$E_p = N^3 / p(N^3/p) = 1$$

Metoda permite obținerea de accelerare și eficiență ideale, fără a lua în calcul costurile suplimentare cu comunicarea și memoria.

Înmulțirea matricilor

Complexitatea (analiză detaliată):

- $\sqrt{p}$ rânduri sunt transmise către cele $\sqrt{p}$ procese. Dimensiunea unui mesaj este n^2/p iar timpul de comunicație este $t_s \log \sqrt{p} + t_w \frac{n^2}{p}\left(\sqrt{p} - 1\right)$
- $\sqrt{p}$ coloane sunt transmise către cele $\sqrt{p}$ procese. Dimensiunea unui mesaj este n^2/p iar timpul de comunicație este $t_s \log \sqrt{p} + t_w \frac{n^2}{p}(\sqrt{p} - 1)$
- Timpul necesar inmulțirii este $\sqrt{p}$ x $(n/\sqrt{p})^3 = n^3/p$
- Timpul rulării în paralel este

$$T_p = \frac{n^3}{p} + 2(t_s \log \sqrt{p} + t_w \frac{n^2}{p}(\sqrt{p} - 1))$$

Înmulțirea matricilor

- Limitarea este dată de consumul mare de memorie.
  - Fiecare proces $P_{i,j}$ folosește $2\sqrt{p}$ blocuri din $\mathbf{A_{i,j}}$ și $\mathbf{B_{i,j}}$
  - Fiecare proces are nevoie de $\Theta(n^2/\sqrt{p})$ memorie
  - Memoria totală necesară pentru toate procesele este $\Theta(n^2 * \sqrt{p})$ ceea ce înseamnă de $\sqrt{p}$ ori mai multă memorie decât algoritmul secvențial.
- Algoritmul este considerat eficient în condițiile în care costurile suplimentare (două broadcast-uri, unul pentru distribuirea datelor la procese și unul pentru agregarea rezultatelor) sunt mici datorită lățimii de bandă și a concurenței.

Inmulțirea matricilor: Algoritmul lui Cannon

Scop: Să îmbunătățească eficiența memoriei consumate de algoritmul clasic cu blocuri.

Abordare:

- Mută blocurile pas cu pas în $\sqrt{p}$ pași (*p* este numărul de procesoare).
- Transmite fiecare bloc (en., *chunk*) din matricile A și B între procesoarele care se află pe același rând sau coloană.
  - Transmiterea se face circular
- Modul de transmitere asigură caracterul distribuit al algoritmului.
- Propus de Lynn Elliot Cannon

Cannon, L. E. (1969). *A cellular computer to implement the Kalman filter algorithm* (Doctoral dissertation, Montana State University-Bozeman, College of Engineering).

Inmulțirea matricilor: Algoritmul lui Cannon

- **Important**: Pentru determinarea submatricii $C_{i,j}$ este nevoie de TOATE submatricile $A_{i,k}$ și $B_{k,j}$ pentru $0 \leq k < \sqrt{p}$

- Formula care evită disputa pentru accesul comultan la același bloc din matricile A și B este:

$$C_{i,j} = \sum_{k=0}^{\sqrt{p}-1} A_{i,(i+j+k)\%\sqrt{p}} * B_{(i+j+k)\%\sqrt{p},j}$$

Inmulțirea matricilor: Algoritmul lui Cannon

//**Alinierea inițială**: Procesul $P_{i,j}$ va înmulți submatricile primite pentru a
// determina un prim rezultat inițial pentru $C_{i,j}$

for i, j = 0 **to** $\sqrt{p} - 1$ **do**

Trimite blocul $A_{i,j}$ procesului $[i, (j - i + \sqrt{p}\,)\ mod\ \sqrt{p}]$ și

blocul $B_{i,j}$ procesului $[(i - j + \sqrt{p}\,) mod\ \sqrt{p}, j]$;

endfor;

Matricea inițială A
Rândul 0 este neschimbat
Rândul 1 este deplasat cu o poziție la stânga
Rândul 2 este deplasat cu 2 poziții la stânga
Rândul 3 este deplasat cu 3 poziții la stânga
...

Matricea inițială B
Coloana 0 este neschimbată
Coloana 1 este deplasată cu o poziție în sus
Coloana 2 este deplasată cu 2 poziții în sus
Coloana 3 este deplasatăcu 3 poziții în sus
...

Inmulțirea matricilor: Algoritmul lui Cannon

//**Calculează-și-transferă**: Realizează o succesiune de pași care constă

// dintr-un transfer (en., *shift*) și un calcul pentru perechile $A_{i,k}$ și $B_{k,j}$
//din procesul $P_{i,j}$

for step = 1 **to** $\sqrt{p} - 1$ **do**

Transferă blocul $A_{i,j}$ cu cel din dreapta (i.e., shift la stânga) și

Transferă blocul $B_{i,j}$ cu cel de jos (i.e., shift în sus)

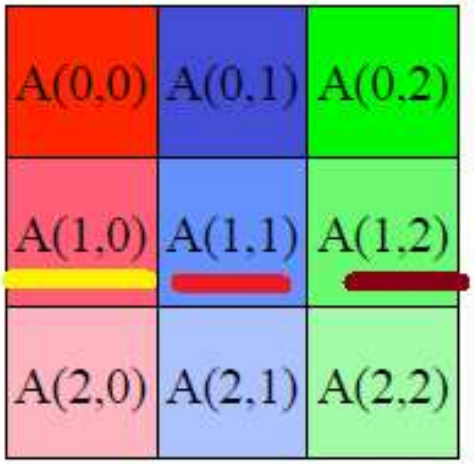
Procesul Pi,j **calculează** submatricea Ci,j prin înmulțirea matricelor primite și adunarea cu valoarea inițială

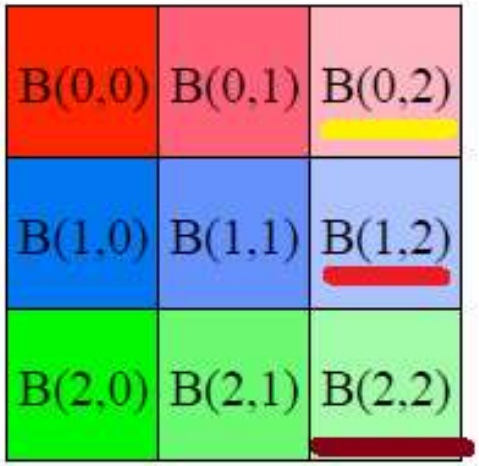
endfor;

Inmulțirea matricilor: Algoritmul lui Cannon

Exemplu pentru matrice de deminesiune 3x3

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

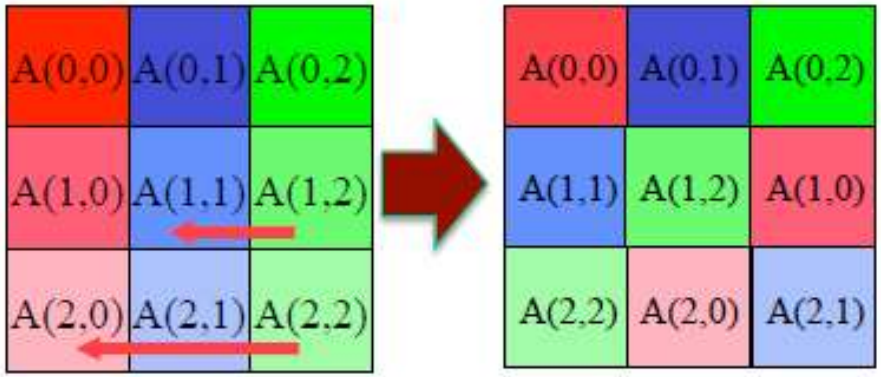


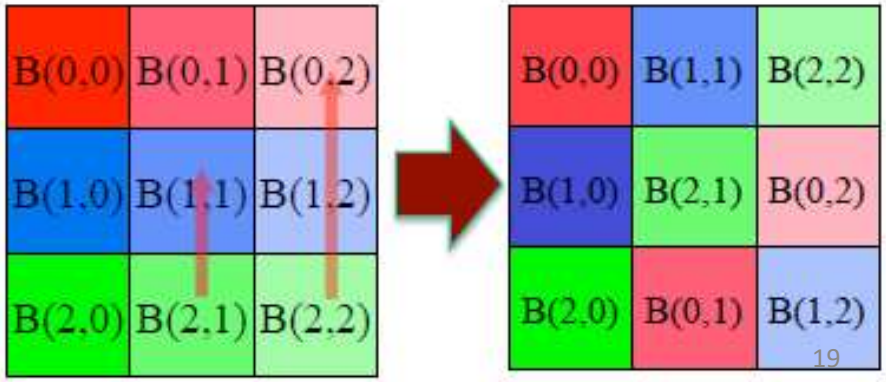


Inmulțirea matricilor: Algoritmul lui Cannon

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

- Initial trebuie ca A[1,0] și B[0,2] să se afle pe același procesor.
- Transferă (en., *shift*) rândurile și coloanele astfel încât valorile blocurilor A[1,1] și B[1,2] să fie aliniate.
- Asemănător se efectuează transferul pentru A[1,2] și B[3,2].



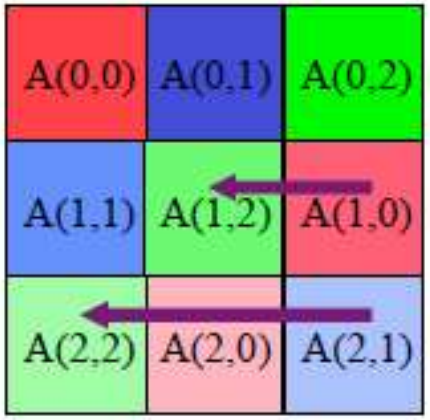


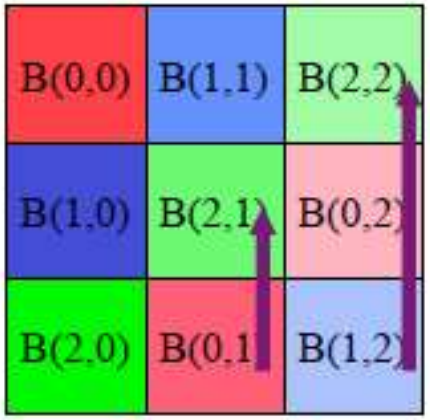
Inmulțirea matricilor: Algoritmul lui Cannon

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

Alinierea matricilor:

- Matricile trebuie inițial aliniate astfel încât primele blocuri necesare (A[1,0] și B[0,2]) să se afle la acelați procesor P[1,2] care va determina valoarea blocului C[1,2].
- Alinierea se realizează prin transferul (en., *shift*) fiecărui rând *i* cu *i* poziții la stânga.
- Același mecanism se folosește și pentru coloane.



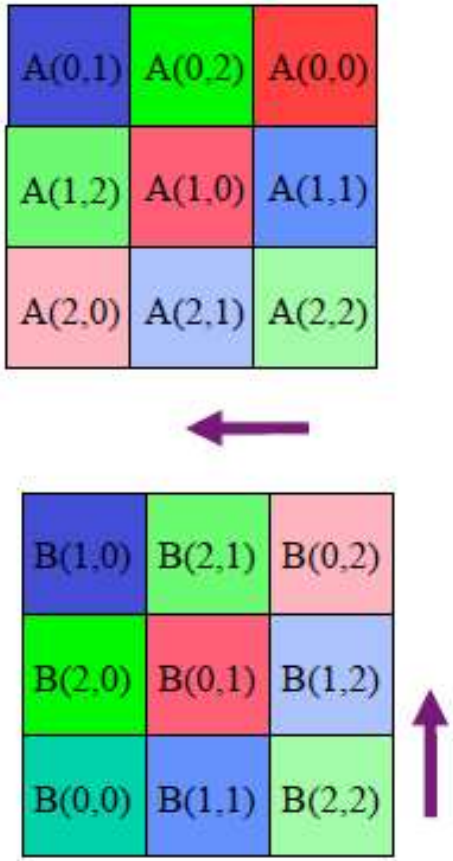


Inmulțirea matricilor: Algoritmul lui Cannon

C[1,2] = A[1,0]*B[0,2] + A[1,1]*B[1,2] + A[1,2]*B[2,2]

Transferă și calculează:

- Este nevoie de $\sqrt{p}$ pași.
- Transferul este circular (en., *circularly shift*)
  - Fiecare rând cu o poziție la stânga
  - Fiecare coloană cu o poziție în sus
- Fiecare procesor efectuează calculul local al înmulțirii blocurilor primite și adună rezultatul la valoarea curentă.



Înmulțirea matricilor : Algoritmul lui Cannon

- Accelerarea și eficiența

$$S_p = N^3 / (N^3/p) = p$$

$$E_p = N^3 / p(N^3/p) = 1$$

Metoda permite obținerea de accelerare și eficiență ideale, fără a lua în calcul costurile suplimentare cu comunicarea și memoria.

Inmulțirea matricilor: Algoritmul lui Cannon

Performanța algoritmului (analiză detaliată)

- Alinierea inițială
  - Distanța maximă a transferului este $\sqrt{p}$ -1
  - Toate operațiile de transfer circular ale rândurilor și coloanelor necesită

$$t_{comm} = 2(t_s + \frac{t_w n^2}{p})$$

- Transferă și calculează
  - Fiecare dintre cei $\sqrt{p}$ pași necesită $t_s + \frac{t_w n^2}{p}$ pentru realizarea transferurilor
  - Înmulțirea a $\sqrt{p}$ submatrici de dimensiune (n/ $\sqrt{p}$)x(n/ $\sqrt{p}$) necesită n^3/p timp.
- Timpul de execuție paralel este

$$T_p = \frac{n^3}{p} + 2\sqrt{p}\left(t_s + \frac{t_w n^2}{p}\right) + 2\left(t_s + \frac{t_w n^2}{p}\right)$$

Inmulțirea matricilor: Algoritmul lui Cannon

Limitări:

- *p* trebuie să fie pătrat perfect
- A și B trebuie să fie matrici pătrate (*n* x *n*), iar n trebuie să fie divizibil cu $\sqrt{p}$
- Integrarea intr-o aplicație sau cu alte biblioteci este dificilă sau are costuri ridicate din cauza comunicației necesare.

Inmulțirea matricilor: Algoritmul lui Fox

- Matricile A și B de dimensiune *n* x *n* sunt distribuite între cele *p* procesoare, astfel încât fiecare procesor deține inițial o submatrice de dimensiune $\left(\frac{\mathrm{n}}{\sqrt{p}}\right) \times \left(\frac{\mathrm{n}}{\sqrt{p}}\right)$
- Algoritmul distribuie *one-to-all* blocurile din matricea A către procesoarele de pe rânduri, și realizează deplasări circulare în sus pentru blocurile matricii B de-a lungul procesoarelor de pe coloane.
- Inițial, fiecare bloc $A_{i,\,i}$ este selectat pentru distribuire.

Algoritmul lui Fox

- Repetă de $\sqrt{p}$ ori
    - Distribuie $A_{i,\,i}$ către toate procesoarele aflate pe același rând.
    - Înmulțeste blocul A primit cu blocul corespondent din B.
    - Trimite blocul B la procesorul de deasupra (circular).
    - Selectează blocul $A_{i,\,(j+1)\%\sqrt{p}}$ (unde $A_{i,\,j}$ este blocul transmis la pasul anterior) și îl transmite către toate procesoarele din acel rând.

Algoritmul lui Fox: Descompunerea domeniului

- Un procesor rulează un subtask (i, j) pentru a determina elementul (sau blocul) $\mathbf{C_{ij}}$ din matricea rezultat C. Astfel, peocesoarele/subtask-urile formează un grid 2D de dimensiune q x q ($q = \frac{n}{\sqrt{p}}$)
- Fiecare procesor/subtask gestionează 4 elemente/blocuri:
  - Elementul/blocul C_{ij} din matricea rezultat, acesta este ceea ce calculează procesorul/subtask-ul.
  - Blocul/elementul Aij din matricea A, transmis procesului/subtaskului înainte de efectuarea calculelor.
  - Blocurile A_{ij}' și B_{ij}' din matricile A și B, primite de proces/subtask în timpul efectuării calculelor.

Algoritmul lui Fox: Exmplificare intuitivă

Inițializare:

Elementele A_{ii} sunt distribuite către matricea rezultat C astfel:

$\mathbf{C_{0,0}} = A_{0,0} * B_{0,0}$; $\mathbf{C_{0,1}} = A_{0,0} * B_{0,1}$; $\mathbf{C_{0,2}} = A_{0,0} * B_{0,2}$; $\mathbf{C_{0,3}} = A_{0,0} * B_{0,3}$

$\mathbf{C_{1,0}} = A_{1,1} * B_{1,0}$; $\mathbf{C_{1,1}} = A_{1,1} * B_{1,1}$; $\mathbf{C_{1,2}} = A_{1,1} * B_{1,2}$; $\mathbf{C_{1,3}} = A_{1,1} * B_{1,3}$

$\mathbf{C_{2,0}} = A_{2,2} * B_{2,0}$; $\mathbf{C_{2,1}} = A_{2,2} * B_{2,1}$; $\mathbf{C_{2,2}} = A_{2,2} * B_{2,2}$; $\mathbf{C_{2,3}} = A_{2,2} * B_{2,3}$

$\mathbf{C_{3,0}} = A_{3,3} * B_{3,0}$; $\mathbf{C_{3,1}} = A_{3,3} * B_{3,1}$; $\mathbf{C_{3,2}} = A_{3,3} * B_{3,2}$; $\mathbf{C_{3,3}} = A_{3,3} * B_{3,3}$

A_{00} B_{00}	B_{01}	B_{02}	B_{03}
B_{10}	A_{11} B_{11}	B_{12}	B_{13}
B_{20}	B_{21}	A_{22} B_{22}	B_{23}
B_{30}	B_{31}	B_{32}	A_{33} B_{33}

Algoritmul lui Fox: Exemplificare intuitivă (cont.)

Pasul 1: Se transmit urmatoarele elemente din A (i.e., cele din dreapta diagonalei principale) către procesoarele corespunzătoare, coloanele din B sunt shiftate cu o poziție în sus, și se realizează înmulțirea.

B_{10}	A_{01} B_{11}	B_{12}	B_{13}
B_{20}	B_{21}	A_{12} B_{22}	B_{23}
B_{30}	B_{31}	B_{32}	A_{23} B_{33}
A_{30} B_{00}	B_{01}	B_{02}	B_{03}

$\mathbf{C_{0,0}} \mathrel{+}= A_{0,1} * B_{1,0}$; $\mathbf{C_{0,1}} \mathrel{+}= A_{0,1} * B_{1,1}$; $\mathbf{C_{0,2}} \mathrel{+}= A_{0,1} * B_{1,2}$; $\mathbf{C_{0,3}} \mathrel{+}= A_{0,1} * B_{1,3}$

$\mathbf{C_{1,0}} \mathrel{+}= A_{1,2} * B_{2,0}$; $\mathbf{C_{1,1}} \mathrel{+}= A_{1,2} * B_{2,1}$; $\mathbf{C_{1,2}} \mathrel{+}= A_{1,2} * B_{2,2}$; $\mathbf{C_{1,3}} \mathrel{+}= A_{1,2} * B_{2,3}$

$\mathbf{C_{2,0}} \mathrel{+}= A_{2,3} * B_{3,0}$; $\mathbf{C_{2,1}} \mathrel{+}= A_{2,3} * B_{3,1}$; $\mathbf{C_{2,2}} \mathrel{+}= A_{2,3} * B_{3,2}$; $\mathbf{C_{2,3}} \mathrel{+}= A_{2,3} * B_{3,3}$

$\mathbf{C_{3,0}} \mathrel{+}= A_{3,0} * B_{0,0}$; $\mathbf{C_{3,1}} \mathrel{+}= A_{3,0} * B_{0,1}$; $\mathbf{C_{3,2}} \mathrel{+}= A_{3,0} * B_{0,2}$; $\mathbf{C_{3,3}} \mathrel{+}= A_{3,0} * B_{0,3}$

Algoritmul lui Fox: Exemplificare intuitivă (cont.)

Pasul 2: Se transmit urmatoarele elemente din A către procesoarele corespunzătoare, coloanele din B sunt shiftate cu o poziție în sus, și se realizează înmulțirea.

$\mathbf{C_{0,0}}$ += $A_{0,2}$* $B_{2,0}$; $\mathbf{C_{0,1}}$ += $A_{0,2}$* $B_{2,1}$; $\mathbf{C_{0,2}}$ += $A_{0,2}$* $B_{2,2}$; $\mathbf{C_{0,3}}$ += $A_{0,2}$* $B_{2,3}$

$\mathbf{C_{1,0}}$ += $A_{1,3}$* $B_{3,0}$; $\mathbf{C_{1,1}}$ += $A_{1,3}$* $B_{3,1}$; $\mathbf{C_{1,2}}$ += $A_{1,3}$* $B_{3,2}$; $\mathbf{C_{1,3}}$ += $A_{1,3}$* $B_{3,3}$

$\mathbf{C_{2,0}}$ += $A_{2,0}$* $B_{0,0}$; $\mathbf{C_{2,1}}$ += $A_{2,0}$* $B_{0,1}$; $\mathbf{C_{2,2}}$ += $A_{2,0}$* $B_{0,2}$; $\mathbf{C_{2,3}}$ += $A_{2,0}$* $B_{0,3}$

$\mathbf{C_{3,0}}$ += $A_{3,1}$* $B_{1,0}$; $\mathbf{C_{3,1}}$ += $A_{3,1}$* $B_{1,1}$; $\mathbf{C_{3,2}}$ += $A_{3,1}$* $B_{1,2}$; $\mathbf{C_{3,3}}$ += $A_{3,1}$* $B_{1,3}$

B_{20}	B_{21}	A_{02} B_{22}	B_{23}
B_{30}	B_{31}	B_{32}	A_{13} B_{33}
A_{20} B_{00}	B_{01}	B_{02}	B_{03}
B_{10}	A_{31} B_{11}	B_{12}	B_{13}

Algoritmul lui Fox: Exemplificare intuitivă (cont.)

Pasul 3: Se transmit urmatoarele elemente din A către procesoarele corespunzătoare, coloanele din B sunt shiftate cu o poziție în sus, și se realizează înmulțirea.

$\mathbf{C_{0,0}}$ += $A_{0,3}$* $B_{3,0}$; $\mathbf{C_{0,1}}$ += $A_{0,3}$* $B_{3,1}$; $\mathbf{C_{0,2}}$ += $A_{0,3}$* $B_{3,2}$; $\mathbf{C_{0,3}}$ += $A_{0,3}$* $B_{3,3}$

$\mathbf{C_{1,0}}$ += $A_{1,0}$* $B_{0,0}$; $\mathbf{C_{1,1}}$ += $A_{1,0}$* $B_{0,1}$; $\mathbf{C_{1,2}}$ += $A_{1,0}$* $B_{0,2}$; $\mathbf{C_{1,3}}$ += $A_{1,0}$* $B_{0,3}$

$\mathbf{C_{2,0}}$ += $A_{2,1}$* $B_{1,0}$; $\mathbf{C_{2,1}}$ += $A_{2,1}$* $B_{1,1}$; $\mathbf{C_{2,2}}$ += $A_{2,1}$* $B_{1,2}$; $\mathbf{C_{2,3}}$ += $A_{2,1}$* $B_{1,3}$

$\mathbf{C_{3,0}}$ += $A_{3,2}$* $B_{2,0}$; $\mathbf{C_{3,1}}$ += $A_{3,2}$* $B_{2,1}$; $\mathbf{C_{3,2}}$ += $A_{3,2}$* $B_{2,2}$; $\mathbf{C_{3,3}}$ += $A_{3,2}$* $B_{2,3}$

B_{30}	B_{31}	B_{32}	A_{03} B_{33}
A_{10} B_{00}	B_{01}	B_{02}	B_{03}
B_{10}	A_{21} B_{11}	B_{12}	B_{13}
B_{20}	B_{21}	A_{32} B_{22}	B_{23}

Algoritmul lui Fox: Complexitate

- Algoritmul lui Fox este efficient d.p.d.v. al memoriei necesare
- Totuși, costurile suplimentare apărute din cauza comunicării sunt mai mari decât cele necesare Algoritmului lui Cannon.
- Scalarea și distribuirea/alocarea subtask-urilor pe procesoare
  - Dimeniunile matricelor pot fi alese astfel încât numărul de subtask-uri este același cu numprul de procesoare.
  - Execuția cea mai eficientă pentru rularea în paralel se obține atunci când topologia rețelei este un grid 2D. Astfel, subtask-ul (i,j) este alocat procesorului $\mathbf{p_{ij}}$. In această situație, accelerarea și eficiența sunt ideale

$$S_p = n^2 / (n^2/p) = p, \text{ si } E_p=n^2/(p*(n^2/p)) = 1$$

fără a lua în calcul costurile suplimentare cu comunicarea și memoria.

Algoritmul SUMMA (en., *Scalable Universal Matrix Multiplication Algorithm*)

- Fie A[m x n] * B[k x n] = C[m x n]

Inmulțirea naivă:	**Dacă schimbăm ordinea?**

For i = 0 to n
  For j = 0 to n
  For k = 0 to n
  C[i,j] += A[i,k] * B[k,j]
	For k = 0 to n
	  For i = 0 to n
	  For j = 0 to n
	  C[i,j] += A[i,k]*B[k,j]

Calculăm n^2 valori (en., *inner products*) C[i,j] = A[i,:]*B[:,j]	Calculăm n valori (en., *outer products*)

Algoritmul SUMMA

- Presupunem ca avem procesoarele distribute într-un grig, P(i,j)
- Exemplu de grid 2 x 2

P(1,1)	P(1,2)
P(2,1)	P(2,2)

A

x

P(1,1)	P(1,2)
P(2,1)	P(2,2)

B

=

P(1,1)	P(1,2)
P(2,1)	P(2,2)

C

Dacă A[mxk] si B[kxn] => C[mxn]

- Fiecare processor va realiza k înmulțiri

Q: Cum administrăm/tratăm/realizăm comunicația?

Algoritmul SUMMA

Fiecare proces P(i,j) execută

- Pentru k = 0 la n-1
  - Se transmite (broadcast) coloana k din A (a_i) rândului i [1]
  - Se transmite (broadcast) rândul k din B (b_j) coloanei j [2]
  - C += a_i x b_j;// outer product [3]

Van De Geijn, R. A., & Watts, J. (1997). SUMMA: Scalable universal matrix multiplication algorithm. *Concurrency: Practice and Experience*, *9*(4), 255-274.

http://www.netlib.org/lapack/lawnspdf/lawn96.pdf

Algoritmul SUMMA

Care sunt costurile de comunicație?

- Fie α = costul necesar transmiterii unui mesaj și β = lățimea de bandă.
- Execuția [1] între p procese durează **log(p)*($\boldsymbol{\alpha + \beta s}$)**, unde s este dimensiunea mesajului.
- Pentru fiecare k, sunt necesare două transmiteri: una pe coloane și una pe rânduri.
- Fiecare coloană parțială trimisă are dimensiunea m/r și fiecare rând parțial trimis are dimenisunea n/c.
- Costul de comunicație total = k*(log(c)(α+β*m/r)+log(r)(α+β*n/c))
  - Intuitiv, trebuie transmise foarte multe mesaje!
- Soluție: În loc sa transmitem rânduri și coloane, transmitem blocuri de rânduri și coloane.
  - Aceeași cantitate de date este transmisă, dar mai puține mesaje => costuri suplimentare mai mici.

OpenACC: https://www.openacc.org/

- OpenACC 1.0 in November of 2011
- OpenACC 3.0 was released in November 2019
- Mod facil de a obține accelerarea aplicațiilor
  - Alte variante:
    - Cu ajutorul bibliotecilor: ”drop-in”
    - Cu ajutorul limbajelor de programare: avem flexibilitate maximă

OpenACC

- Directivele OpenACC
  - Funcționează pe GPU many-core si CPU multicore
  - Reprezintă îndicii (en., hints) pentru compilator
  - Compilatorul paralelizează codul
  - Modul de lucru este asemănător cu OpenMP

Recomandări:

- "OpenACC will enable programmers to easily develop portable applications that maximize the performance and power efficiency benefits of the hybrid CPU/GPU architecture of Titan."

  --Buddy Bland, Titan Project Director, Oak Ridge National Lab

- "OpenACC is a technically impressive initiative brought together by members of the OpenMP Working Group on Accelerators, as well as many others. We look forward to releasing a version of this proposal in the next release of OpenMP."

  --Michael Wong, CEO OpenMP Directives Board

OpenACC

- Standardul pentru directive de lucru cu GPU
  - **Ușurința în utilizare**: Directivele sunt ușor de folosit pentru a accelera aplicațiile cu nevoi mari de calcul.
  - **Deschidere** (en., open): OpenACC este standard deschis pentru directive de lucru cu GPU. Acest aspect face ca programarea pe GPU să devină directă și portabilă pe GPU many-core si CPU multicore.
  - **Putere/forță**: Directivele GPU permit acces complet la puterea de procesare paralelă a GPU.

OpenACC

Programare de nivel înalt cu acces la nivel jos

- Directivele compilator specifică proțiunile de cod paralel in C
  - Compilatorul OpenACC rulează porțiunile de cod paral pe GPU și nu pe sistemul gazdă
  - Sunt portabile peste OSs, sistemele gazdă, acceleratoare și compilatoare
- Permit obținerea de programe eterogene de nivel înalt
  - Fără inițializarea explicită a acceleratorului
  - Fără să fie nevoie de tranferul explicit de codsau date între calculatorul gazdă și accelerator.
- Sunt compatibile cu alte limbaje de lucru cu GPU și biblioteci
  - Interoperează cu CUDA C/Fortran și alte bibliotecu GPU (i.e., CUFFT, CUBLAS, CUSPARSE, etc.)

OpenACC

Avantaj: ”Optimizing code with directives is quite easy, especially compared to CPU threads or writing CUDA kernels. The most important thing is avoiding restructuring of existing code for production applications. ”

- Developer at the Global Manufacturer of Navigation Systems

- Aspectul principal în dezvoltare se referă la specificarea/expunerea paralelismului prin folosirea directivelor.

OpenACC: Exemplu

```
void saxpy(int n, float a, float *x, float *restrict y)
{
#pragma acc kernels
 for (int i = 0; i < n; ++i)
      y[i] = a*x[i] + y[i];
}
...
// Perform SAXPY on 1M elements
saxpy(1<<20, 2.0, x, y);
...
```

Sintaxa directivelor in C:

```
#pragma acc directive [clause [,] clause] …]
```

…often followed by a structured code block

OpenACC: Exemplu (cont.)

Kernels: primele directive OpenACC

//Fiecare buclă se execută ca un kernel separat pe GPU.

```
!$acc kernels
   do i=1,n
         a(i) = 0.0
         b(i) = 1.0
         c(i) = 2.0
   end do

   do i=1,n
         a(i) = b(i) + c(i)
   end do
!$acc end kernels
```

Kernel #1

Kernel #2

Kernel =
o funcție care rulează în paralel pe GPU

OpenACC: Exemplu (cont.)

- Construirea de Kernels in C

```
#pragma acc kernels [clause …]
 { structured block }
```

Clauses:

```
        if( condition )
        async( expression )
        // clauze care se referă la date
```

OpenACC: Exemplu (cont.)

- Cuvântul cheie *restrict*
  - Reprezintă declararea intenției programatorului către compilator
  - Ex., aplicat la un pointer float *restrict ptr;
- Înseamnă: ”pe durata de viață a variabilei **ptr** , numai aceasta sau o valoare obținută din aceasta (ex., **ptr+1**) va putea fi folosită pentru a accesa obiectul către care pointează variabila”.
- Implicații:
  - Se limitează efectele de *pointer aliasing*.
- Compilatoarele OpenACC necesită desori folosirea cuvântului cheie restrict pentru a determina independența
  - În caz contrar compilatorul nu poate paraleliza buclele care accesează **ptr**.
  - Dacă programatorul nu definesc corect independența => *undefined behaviour*

OpenACC: Exemplu (cont.)

Variabila **y** nu va fi un pointer alias pentru x.

```
#include <stdlib.h>
void saxpy(int n, float a,
float *x, float *restrict y)
{
#pragma acc kernels
 for (int i = 0; i < n; ++i)
 y[i] = a*x[i] + y[i];
}
```

```
int main(int argc, char **argv)
{
 int N = 1<<20; // 1 million floats
if (argc > 1)
        N = atoi(argv[1]);
float *x = (float*)malloc(N * sizeof(float));
float *y = (float*)malloc(N * sizeof(float));
for (int i = 0; i < N; ++i) {
        x[i] = 2.0f;
         y[i] = 1.0f;
}
saxpy(N, 3.0f, x, y);
return 0;
}
```

OpenACC: Exemplu (cont.)

- Compilare și rulare în C:

pgcc –acc [-Minfo=accel] –o saxpy_acc saxpy.c

```
saxpy:
 8, Generating copyin(x[:n-1])
 Generating copy(y[:n-1])
 Generating compute capability 1.0 binary
 Generating compute capability 2.0 binary
 9, Loop is parallelizable
 Accelerator kernel generated
 9, #pragma acc loop worker, vector(256) /* blockIdx.x threadIdx.x */
 CC 1.0 : 4 registers; 52 shared, 4 constant, 0 local memory bytes; 100% occupancy
 CC 2.0 : 8 registers; 4 shared, 64 constant, 0 local memory bytes; 100% occupancy
```

OpenACC: Exemplu (cont.)

- Clauze pentru date

copy (list) : Alocă memorie pe GPU și copiază date care se află pe sistemul gazdă pe GPU atunci când se intră în porțiunea paralelizată, și transferă datele înapoi pe sistemul gazdă la ieșire din zona paralelizată.

copyin (list) Alocă memorie pe GPU și copiază date care se află pe sistemul gazdă pe GPU atunci când se intră în porțiunea paralelizată.

copyout (list) Alocă memorie pe GPU și copiază date din GPU pe sistemul gazdă când se iese în porțiunea paralelizată.

create (list) Alocă memorie pe GPU dar nu copiază date.

present (list) Datele sunt deja prezente pe GPU.

OpenACC: alte clauze și directive

- Array shaping: atunci când nu se poate determina dimensiunea unui array.
- Update: #pragma acc update [clause …]

  Clauze:

  host(list) if(expression)

  device(list) async(expression)

  - Pentru a updata date după ce o copie a acestora a fost modificată (ex., updatarea copiei din GPU dupa ce copia de pe sistemul gazdă a fost modificată).
  - Pentru mutarea datelor între GPU și gazda.

OpenACC: Determinarea oportunităților de paralelizare

- Buclele încuibate (en., nested for loops)
- Corpurile buclelor trebuie să fie **independente** unele de altele
  - Compilatorul poate fi ajutat cu restrict sau clauza independent
- Compilatorul trebuie să poată determina dimensiuena datelor utilizate în procesare.
  - Se pot folosi directive care controlează explicit dimensiunea
- Aritmetica pointerilor trebuie evitată
  - Folosiți accesarea cu indecși în loc de cea cu pointeri.
- Funcțiile apelate în cadrul secțiunii accelerate trebuie sa fie *inline*.